Baseline vs. Event-Driven Autoscaling in Kubernetes: A Controlled Comparison of HPA and KEDA for a Queue-Backed Microservice

Group 5

CS5296 Cloud Computing, Spring 2026, City University of Hong Kong

April 2026

Abstract

Cloud-native microservices rely on elastic scaling to absorb volatile workloads. Kubernetes ships two commonly used autoscalers: the resource-based Horizontal Pod Autoscaler (HPA) and the event-driven Kubernetes Event-Driven Autoscaling (KEDA) framework. We conduct a controlled comparative study of the two on a RabbitMQ-backed Spring Boot consumer deployed on a single-node K3s cluster hosted on an AWS EC2 `m5.large` instance. Under an identical flash-crowd workload (a 10 s burst targeting 1 000 messages/s), three replications per group are collected and compared along three axes: responsiveness, user-visible effect, and resource utilisation. All non-signal scaling parameters are held constant—in particular the `scaleDown` `stabilizationWindowSeconds` is fixed at 60 s and the `scaleDown` rate policy at 50 %/60 s for both groups. Under this configuration, KEDA (queueLength signal) reacts 10.4 s faster than Baseline HPA (CPU signal) on average (63.4 ± 3.4 s versus 73.8 ± 8.3 s), drains the backlog 12.2 s sooner, and begins to scale down ≈ 327 s after the burst ends, whereas the Baseline HPA does not release any capacity during the entire 355 s post-burst trial window. The root cause is the signal's information asymmetry: CPU utilisation remains pinned at the `limits.cpu` ceiling throughout the drain, structurally preventing HPA from inferring a reduced workload, while queue length decreases monotonically and thus serves as a leading indicator. Within the observed window both groups still hit the same peak of six pods and have near-identical pod-seconds overshoot, so KEDA's concrete resource advantage is earlier capacity release rather than a lower peak replica count. For queue-backed event-driven workloads, queue-length-based autoscaling delivers measurable quality-of-service gains that parameter tuning alone cannot recover.

I Introduction

Elasticity is a defining property of cloud computing [1]: cloud-hosted applications acquire additional compute capacity when demand rises and release it when demand falls. In contemporary asynchronous architectures, that demand typically arrives through message queues—for example, the order-ingestion pipelines behind e-commerce flash sales, telemetry fan-in from Internet-of-Things fleets, or job bursts in machine-learning inference back-ends. Whenever the

backlog grows, the service consuming those messages must scale out *before* the queue becomes deep enough to breach service-level objectives.

Kubernetes [2] has become the de facto container orchestrator and ships with the *Horizontal Pod Autoscaler* (HPA) [3], which adjusts a Deployment's replica count so that the observed CPU or memory utilisation approaches an operator-specified target. CPU, however, is a *lagging* indicator for queue-backed workloads: by the time it crosses the scaling threshold, the backlog may already be large, and in resource-capped pods the reported utilisation can be driven entirely by the `limits.cpu`/`requests.cpu` ratio rather than by the actual workload. Kubernetes Event-Driven Autoscaling (KEDA) [4], a Cloud Native Computing Foundation (CNCF) graduated project, addresses this limitation by allowing operators to scale directly on business metrics such as RabbitMQ queue length. Internally KEDA materialises a hidden HPA and feeds it the external metric, so the underlying control loop is identical—only the signal differs.

Research question. *Given an identical queue-backed consumer under a flash-crowd workload, how do Baseline HPA (CPU signal) and KEDA (queueLength signal) differ along three axes—responsiveness, user-visible effect, and resource utilisation—once all other scaling parameters are held equal?*

Contributions. This paper makes three contributions:

- A reproducible benchmarking harness, open-sourced on GitHub [5], that provisions a K3s cluster on a single AWS EC2 `m5.large` node, deploys two identical Spring Boot consumers, and injects a configurable burst workload via RabbitMQ.
- A controlled comparison of Baseline HPA and KEDA across three metric axes, covering both scale-up and scale-down behaviour, with every experimental variable fixed except the scaling signal.
- A quantitative explanation of why the Baseline HPA cannot shrink the replica set within the trial window: the CPU signal remains pinned at the pod's `limits.cpu` throughout the drain and therefore does not descend until *after* the queue is empty.

Roadmap. Section II reviews the HPA and KEDA control loops. Section III describes our system design. Section IV documents the experimental methodology. Section V presents the measurements. Section VI discusses practitioner implications, and Section VII concludes.

II Background and Related Work

II-A Horizontal Pod Autoscaler

The Kubernetes HPA [3] runs a reconciliation loop with period controlled by the `-horizontal-pod-autoscale` flag (15 s by default). At each tick it queries `metrics-server`—itself refreshed every 15 s from `kubelet`/cAdvisor—for the target metric and computes the desired replica count via

$$n_{\text{desired}} = \left\lceil n_{\text{current}} \cdot \frac{m_{\text{current}}}{m_{\text{target}}} \right\rceil, \tag{1}$$

where n_{desired} is further clamped to the $[\text{minReplicas}, \text{maxReplicas}]$ range. A `behavior` block governs how quickly the desired change takes effect: `stabilizationWindowSeconds` computes the running maximum over a rolling window (so transient dips do not immediately trigger scale-down), while a `policies` list such as `Percent 50%/60s` caps the per-period scaling rate. When the signal is `cpu.Utilization`, m_{current} is the mean CPU usage divided by `requests.cpu`. Consequently, a pod whose CPU is pinned at `limits.cpu` reports a utilisation

equal to the *static* limit-to-request ratio rather than the *dynamic* load on the queue it serves—a structural limitation we revisit in Section V.

II-B Kubernetes Event-Driven Autoscaling

KEDA [4] is a CNCF graduated project that extends Kubernetes with the `ScaledObject` custom resource. It ships with more than 60 scalers for message brokers, databases, HTTP endpoints, and custom sources [6], and internally materialises a hidden HPA whose target metric is of type `External` and is served by the KEDA metrics adapter. Thus the control *mechanism* is identical to the standard HPA; the only additional machinery is a short-lived metric pipeline that translates a business signal (for example, RabbitMQ queueLength) into a scaling ratio that Eq. 1 can consume. For a `rabbitmq/QueueLength` trigger the operator specifies a messages-per-replica target T, and KEDA reports $m_{\text{current}}/m_{\text{target}} = Q/T$, where Q is the instantaneous queue depth. The desired replica count therefore simplifies to $\lceil Q/T \rceil$, independent of CPU usage.

II-C Related Work

Prior work spans surveys of reactive and predictive autoscaling algorithms [7], Kubernetes-focused elasticity benchmarks [8, 9], and event-driven or serverless scaling studies [10]. What is less common is a tightly controlled, same-hardware comparison between a CPU-driven HPA baseline and a business-metric-driven controller. The present study narrows the scope deliberately: application, broker, hardware, `behavior` block, and load pattern are held constant, so the only independent variable is the scaling signal itself.

III System Design

III-A Architecture Overview

Fig. 1 depicts the end-to-end deployment. A single AWS EC2 `m5.large` instance (2 vCPU, 8 GiB RAM) hosts a lightweight K3s [11] v1.30 cluster with three application namespaces: `rabbitmq` for the shared broker and `baseline`/`keda` for the two experiment groups. The `keda` namespace also hosts the KEDA operator and metrics adapter installed by Helm. A Python producer running on the evaluator's workstation injects load into either `baseline-queue` or `keda-queue` over AMQP. A shell-based collector (`scripts/collect-metrics.sh`) polls the RabbitMQ Management API [12] and `kubectl` at ≈ 1 Hz and writes per-second comma-separated-value (CSV) rows; the same script runs for both groups so the observation pipeline is identical.

III-B Consumer Application

Both groups run the same Spring Boot [13] consumer image (`htasteqin/cs5296-consumer:v1.0`). Each message triggers a ≈ 200 ms busy-wait so that the workload is CPU-bound. This is a deliberate design choice: a `Thread.sleep` would render HPA blind to the workload—since the only available signal is CPU utilisation—and would therefore bias the comparison before it begins. The pod resource settings are `requests: 100m CPU / 256 MiB` and `limits: 300m CPU / 512 MiB`.

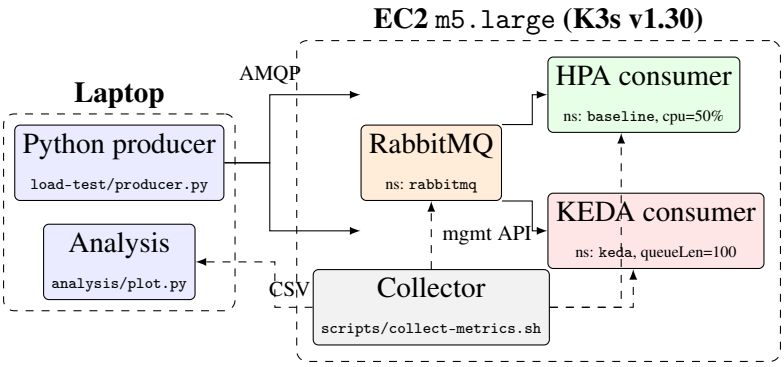

Figure 1. End-to-end architecture on a single AWS EC2 node. The consumer image is shared by both groups; the only variable between them is the scaling policy attached to each Deployment.

III-C Scaling Strategies

The two scalers share identical `minReplicas` = 1 and `maxReplicas` = 6 (dictated by the 2 vCPU node), together with an identical `behavior` block:

- `scaleUp`: `stabilizationWindowSeconds` = 0; policies `Percent 100%/15s` ∪ `Pods 4/15s`, `selectPolicy` = `Max`.
- `scaleDown`: `stabilizationWindowSeconds` = 60 s; policy `Percent 50%/60s`.

Only the scaling *signal* differs: **Baseline HPA** uses `cpu.Utilization` with a 50% target; **KEDA** uses a `rabbitmq` `QueueLength` trigger with target $T = 100$ messages/replica and `pollingInterval` = 5 s.

III-D Control Variables

To isolate the effect of the scaling signal we hold all non-signal parameters constant across the two groups: identical Docker image, identical `requests`/`limits`, identical ≈ 200 ms per-message processing cost, a shared RabbitMQ broker (separate queues per group to prevent interference), the same EC2 node, and the same burst pattern. The only independent variable is therefore the scaling signal—CPU utilisation versus queue length.

IV Experimental Setup

IV-A Environment

- **Cloud**: AWS EC2 `m5.large` (2 vCPU, 8 GiB RAM, `us-west-2`), Ubuntu 22.04 LTS.
- **Cluster**: K3s v1.30 (single-node, embedded SQLite data store), Helm 3, `metrics-server` v0.7, and KEDA v2.13 installed via its official Helm chart.
- **Broker**: a repository-owned RabbitMQ `StatefulSet` using image `rabbitmq:3.12-management`, with `limits.cpu` = 1 000 m, `limits.memory` = 1 GiB, and `tcpSocket:5672` readiness/liveness probes to avoid false failures on the constrained node.

IV-B Workload

The pattern defined in `load-test/patterns/burst.yaml` consists of three phases: a 10 s warm-up at 5 messages/s (so that the collector captures a steady-state baseline at `replicas` = 1), a 10 s burst *targeting* 1 000 messages/s, and 180 s of silence. The harness script `run-experiment.sh` also waits 10 s before launching the producer so the collector can record a baseline, and then appends a further 180 s post-burst observation window. Each trial therefore lasts ≈ 390 s wall-clock, with roughly 355–360 s of observation after the burst ends.

Producer ceiling. The `pika` [14] `BlockingConnection.basic_publish` call is a synchronous RPC that saturates at ≈ 740 msg/s on the `m5.large` node; consequently, the realised burst injects 7 100–7 500 messages rather than the nominal 10 000. The ceiling affects both groups identically—the same producer publishes to both queues with the same pattern—so it does not bias the comparison.

IV-C Metrics and Collection

The collector `scripts/collect-metrics.sh` polls `kubectl` for pod readiness and the RabbitMQ Management API for queue statistics, emitting an 8-column CSV (see artifact appendix). We derive three metric axes:

- **Responsiveness.** Reaction latency (burst start $\rightarrow$ first new pod Ready), scale-up time (burst start $\rightarrow$ peak pod count), and drain time (burst end $\rightarrow$ `queue_depth` = 0).
- **User-visible effect.** Average delivery rate during the drain window and total messages delivered per trial.
- **Resource utilisation.** Scale-down-start time (burst end $\rightarrow$ first pod removal), pods still Ready at the end of the trial (≈ 355 s post-burst), and pod-seconds overshoot, defined as $\int \max\{0, \text{pods}(t) - \text{minReplicas}\}\, dt$ over the entire trial.

IV-D Trial Protocol

For each of the two groups we run $n = 3$ independent trials and report the sample mean with one sample standard deviation. Between trials the Deployment is scaled back to 1 replica and the target queue is purged. Each archived raw CSV therefore corresponds to the same procedure: reset deployment, purge queue, capture a baseline, run the burst pattern, then observe the drain and any scale-down. All six formal trials were executed sequentially on 2026-04-20 and the committed artefacts under `results/raw/` are the ground truth for the statistics reported in this paper.

V Results and Analysis

All numbers in this section are derived from the six trial CSV files in `results/raw/` via `analysis/plot.py`; the full reproduction recipe is documented in the Artifact Appendix. Unless otherwise stated, means are reported with one sample standard deviation over $n = 3$ independent trials per group.

Table I. Headline metrics per group (mean ± std, $n = 3$). "—" indicates that the event did not occur within the trial window (≈ 355 s after burst end).

Metric	Baseline HPA (CPU)	KEDA (queueLength)
Responsiveness		
Reaction latency (s)	73.8 ± 8.3	$\mathbf{63.4 \pm 3.4}$
Scale-up time to peak (s)	90.2 ± 4.6	$\mathbf{80.2 \pm 5.1}$
Queue drain time (s)	309.7 ± 5.2	$\mathbf{297.6 \pm 5.6}$
User-visible effect		
Average throughput during drain (msg/s)	22.75 ± 0.27	$\mathbf{23.35 \pm 0.27}$
Messages delivered per trial	7451 ± 57	7212 ± 90
Resource utilisation		
Scale-down start (s after burst end)	— (not observed)	$\mathbf{326.9 \pm 3.1}$
Ready pods at trial end (≈ burst+355 s)	6.0	**3.0**
Pod-seconds overshoot	1426 ± 27	$\mathbf{1415 \pm 7}$
Peak pod count (`maxReplicas`)	6	6

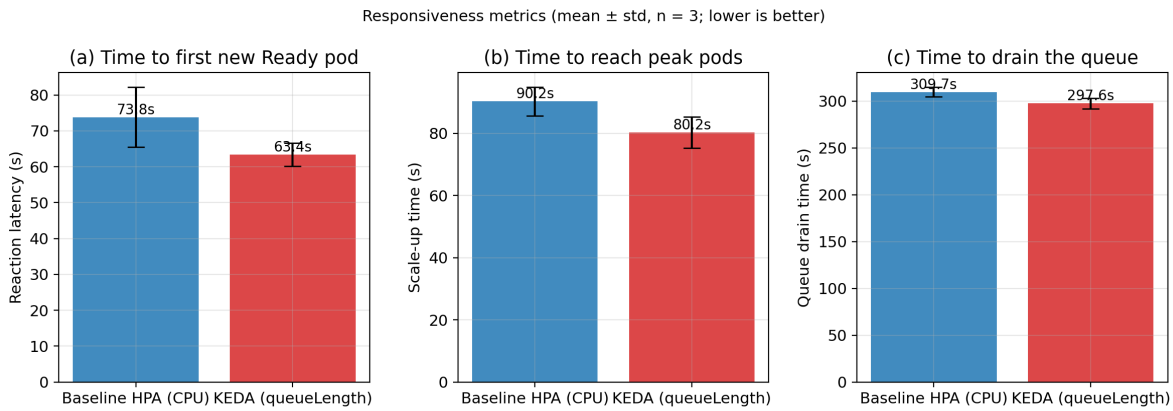

Figure 2. Responsiveness metrics: time to the first new Ready pod, time to peak pods, and time to queue drain. Lower is better; error bars show one sample standard deviation.

V-A Headline Metrics

Table I aggregates every metric we computed. KEDA (queueLength) wins on every responsiveness dimension and is the only scaler that visibly releases capacity during the trial.

Because the realised burst size varies slightly with producer-side publishing speed, `messages_delivered` is descriptive only; the more comparable user-visible outcomes are queue drain time and average throughput, which are normalised over the observed run.

V-B Responsiveness

Fig. 2 summarises the three responsiveness metrics. Relative to the Baseline HPA (CPU), KEDA (queueLength) reacts 10.4 s earlier on average (a 14% reduction), reaches peak replicas 10 s earlier, and drains the backlog 12 s earlier. The standard deviation of the reaction latency is also markedly smaller for KEDA (3.4 s) than for the Baseline HPA (8.3 s): the queue-length signal produces a more predictable response across trials.

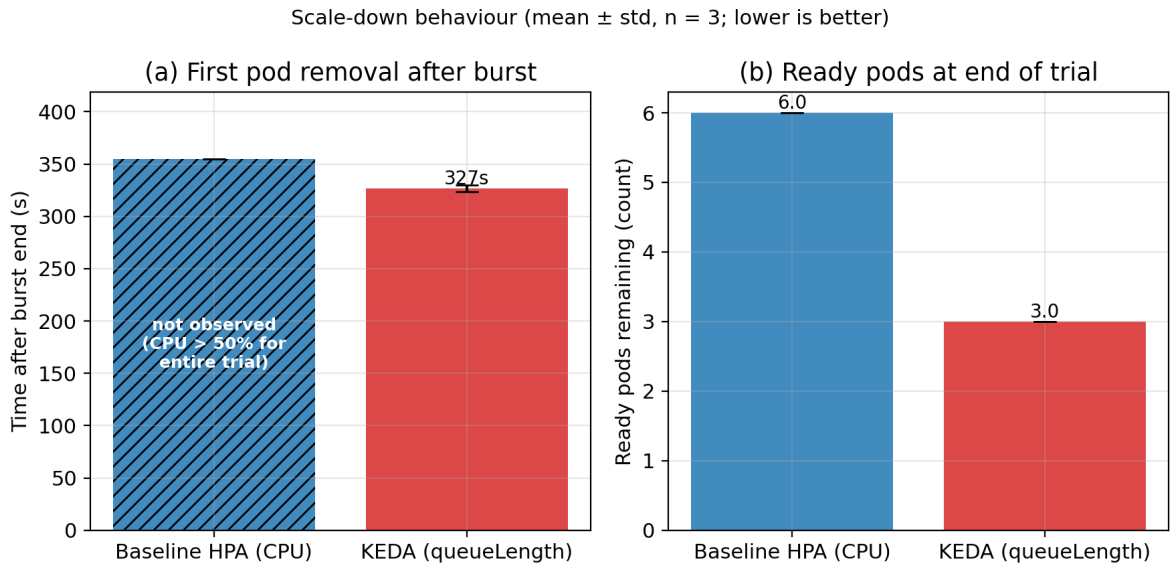

Figure 3. Scale-down behaviour. (a) Elapsed time from burst end to the first pod removal. The hatched bar marks the Baseline HPA's non-event: scale-down did not occur during the trial because the CPU signal remained above the 50% target throughout. (b) Number of Ready pods remaining at the end of the trial.

Per-run evidence. The three paired timeline figures archived under `results/figures/` show the same qualitative pattern in every matched run: KEDA reaches 5–6 Ready pods first (lead: 5–20 s), its queue-depth curve remains below the Baseline HPA during the drain, and it is the only scaler that triggers pod removal before the observation ends. The aggregate bars in Fig. 2 summarise this run-level pattern compactly.

V-C Scale-down Behaviour

Fig. 3 presents the most pronounced contrast of the experiment. Recall from Section III-C that both groups share identical `scaleDown` parameters (`stabilizationWindowSeconds` = 60 s and `Percent 50%/60s`). Despite this parity, the Baseline HPA failed to remove any pod in all three trials, whereas KEDA began shrinking the replica set 326.9 ± 3.1 s after the burst ended and had returned half the fleet by the time the trial concluded.

Why the Baseline HPA cannot shrink. The cause is not the `behavior` block but the scaling *signal*. The consumer is CPU-bound with a 200 ms busy-wait per message, so throughout the drain its CPU usage is pinned at its `limits.cpu` = 300 m. Because the HPA computes utilisation as $m_{\text{current}} = \text{usage}/\text{requests.cpu}$ with `requests.cpu` = 100 m, the reported utilisation is $3\times$ the 50% target for the entire drain phase. Eq. 1 therefore produces n_{desired} saturated at `maxReplicas` = 6 until the queue becomes empty ($\approx$ 310 s post-burst) and each pod falls idle—a transition that occurs only after the $\approx$ 355 s trial window has closed.

Why KEDA shrinks. Under KEDA's queueLength trigger, $n_{\text{desired}} = \lceil Q/T \rceil$ with $T = 100$, so the desired replica count decreases monotonically as Q does. In every trial the queue fell below 500 messages approximately 270 s after burst end, at which point n_{desired} dropped below 6; after the mandatory 60 s stabilisation window the first pod was reaped. KEDA thus captures backlog reduction *while it is in flight*, whereas the Baseline HPA can only observe it *after* the

backlog is gone.

Within-window cost. The pod-seconds overshoot remains close between the two groups ($1\,426 \pm 27$ versus $1\,415 \pm 7$ pod·s, a gap below 1%) because both scalers hit the same peak of six pods and most of the trial is spent draining the burst. The economic advantage of KEDA therefore appears first in the tail: it starts returning pods earlier, but a longer post-drain observation window would be needed for that lead to accumulate into a large overshoot gap.

VI Discussion

The signal matters more than the parameters. The most consequential finding of this study is that KEDA's advantage is rooted in *what* it measures rather than in *how* the underlying HPA control block is configured. Both scalers were given an identical 60 s stabilisation window, an identical `Percent 50%/60s` scale-down policy, and an identical scale-up budget; only the signal differed. Operators tempted to tune the HPA `behavior` block in an attempt to recover KEDA's observed responsiveness are therefore optimising a second-order variable.

Why CPU fails on queue-backed workloads. For a CPU-bound consumer governed by a strict `limits.cpu`, the CPU signal saturates the moment the queue has any residual work: the reported utilisation becomes the ratio of limit to request, independent of the backlog. The HPA can thus neither *estimate* the deficit while the burst is active nor *release* capacity while the drain is still in flight. Queue length, in contrast, is a direct measurement of outstanding work and carries both pieces of information simultaneously.

When HPA remains reasonable. For synchronous HTTP services without an intermediate queue, CPU remains a plausible scaling signal and the HPA's simplicity is attractive. For queue consumers, a hybrid policy is often the pragmatic production choice: let queue length drive normal elasticity and retain CPU-based limits as a guardrail against runaway scale-up.

Limitations and next steps. The study is confined to a single-node cluster, one EC2 instance type, one broker, and three replications per group; it therefore demonstrates a clear directional effect rather than a universal law. The ≈ 355 s post-burst window also truncates the Baseline HPA's scale-down tail, so the observed cost gap is conservative. Future work should sweep `cpu` and queue-length targets, compare against HPA with native `External` metrics, and repeat the study on multi-node clusters.

VII Conclusion

This paper has presented a controlled comparison of Baseline HPA (CPU) and KEDA (queueLength) for a RabbitMQ-backed Spring Boot consumer running on a single-node K3s cluster. With all non-signal parameters held equal over $n = 3$ trials per group, KEDA reacted 10.4 s faster, drained the backlog 12.2 s sooner, and was the only scaler to release capacity during the trial; the Baseline HPA's CPU signal remained pinned at the pod's `limits.cpu` throughout the drain and therefore could not trigger a downscale until after the backlog was gone. For queue-backed microservices, scaling on the business metric is not a preference but a measurable

quality-of-service improvement—independent of how aggressively the HPA `behavior` block is tuned.

References

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep. SP 800-145, 2011.

[2] Cloud Native Computing Foundation, "Kubernetes: production-grade container orchestration," https://kubernetes.io/, 2024, accessed: Apr. 2026.

[3] The Kubernetes Authors, "Horizontal Pod Autoscaling," Kubernetes documentation, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024, accessed: Apr. 2026.

[4] KEDA Maintainers, "KEDA: Kubernetes event-driven autoscaling," https://keda.sh/docs/, 2024, cNCF graduated project. Accessed: Apr. 2026.

[5] CS5296 Group 5, "CS5296-Group5-Autoscaling: reproducibility artifact," https://github.com/QinHaoting/CS5296-Group5-Autoscaling, 2026, mIT-licensed open-source artifact.

[6] KEDA Maintainers, "KEDA scalers," https://keda.sh/docs/latest/scalers/, 2024, accessed: Apr. 2026.

[7] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: a taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, 2018.

[8] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in Kubernetes for elastic container orchestration," *Sensors*, vol. 20, no. 16, p. 4621, 2020.

[9] V. Medel, O. Rana, J. Á. Bañares, and U. Arronategui, "Modelling performance and resource management in Kubernetes," in *Proc. IEEE/ACM 9th Int. Conf. Utility and Cloud Computing (UCC)*, Shanghai, China, 2016, pp. 257–262.

[10] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Annual Technical Conference (ATC)*, 2020, pp. 205–218.

[11] Rancher Labs, "K3s: lightweight Kubernetes," https://k3s.io/, 2024, accessed: Apr. 2026.

[12] Broadcom Inc., "RabbitMQ: reliable and mature messaging and streaming broker," https://www.rabbitmq.com/, 2024, accessed: Apr. 2026.

[13] VMware, Inc., "Spring Boot reference documentation," https://docs.spring.io/spring-boot/, 2024, accessed: Apr. 2026.

[14] Pika Maintainers, "Pika: Python AMQP 0-9-1 client library," https://pika.readthedocs.io/, 2024, accessed: Apr. 2026.

Artifact Appendix

A.1 Abstract

The artifact is a self-contained set of scripts and Kubernetes manifests that (i) provisions a single-node K3s cluster on an AWS EC2 `m5.large` instance, (ii) deploys two identical Spring Boot consumers scaled by the Baseline HPA and by KEDA respectively, (iii) injects a burst workload via a Python producer and records per-second metrics, and (iv) produces six summary figures and a headline CSV. A short shell loop around `scripts/run-experiment.sh` reproduces the six formal trials.

A.2 Check-list

- *Compilation*: Maven 3.9; Docker 24.x (or Colima); `docker buildx` for multi-architecture builds.
- *Run-time*: Ubuntu 22.04 LTS; K3s v1.30; Java 17; Python 3.10+.
- *Hardware*: AWS EC2 `m5.large` (2 vCPU, 8 GiB RAM); `t3.medium` is also viable provided that `maxReplicas` remains ≤ 6.
- *Execution*: $\approx$ 49 min of wall time for the 6 trials, plus 120 s of cooldown between trials.
- *Output*: 12 per-trial CSVs in `results/raw/`, one aggregated `summary.csv`, and six PNG figures in `results/figures/`.
- *Licence*: MIT, publicly available on GitHub.

A.3 How to access

Source code, raw data, and figures: https://github.com/QinHaoting/CS5296-Group5-Autoscaling.

A.4 Installation and deployment

1. Build and push the consumer image:
   ```
   docker buildx build -platform linux/amd64
   -t ⟨dockerhub⟩/cs5296-consumer:v1.0 -push consumer/
   ```
2. On the EC2 node, `bash scripts/setup.sh` installs K3s, Helm, `metrics-server`, KEDA, and the repository-owned RabbitMQ manifests.
3. Apply the two scaling policies:
   ```
   export CONSUMER_IMAGE=⟨image⟩
   bash scripts/deploy-baseline.sh &&
   bash scripts/deploy-keda.sh
   ```

A.5 Experiment workflow

```
# Single trial, either group:
bash scripts/run-experiment.sh baseline 1
bash scripts/run-experiment.sh keda     1
```

```
# Full batch (3 runs per group):
for run in 1 2 3; do bash scripts/run-experiment.sh baseline "$run"; done
for run in 1 2 3; do bash scripts/run-experiment.sh keda "$run"; done

# Offline analysis on the evaluator's laptop:
python analysis/plot.py
```

A.6 Expected result

Six PNG figures (`fig1-run1-timeline.png` through `fig3-run3-timeline.png` for per-run timelines, and `fig4-response-bar.png`, `fig5-scale-down-bar.png`, `fig6-overshoot-cost.png` for the aggregates), together with a `summary.csv` containing 10 metrics per trial. KEDA reacts $\approx$ 10 s faster than the Baseline HPA, drains the queue $\approx$ 12 s sooner, and is the only scaler to trigger scale-down during the trial.

A.7 Troubleshooting

Full diagnostics appear in `docs/ARTIFACT.md` and in the detailed execution manual under the course planning folder (`Group/plan/`), especially its execution log and formal experiment narrative appendices.